

Leer CSV/Excel en Oracle PL/SQL Developer y guardarlo en una tabla

Te doy un panorama completo con **6 enfoques**, ordenados de mejor a peor según tu escenario (Windows, Oracle DB, PL/SQL Developer). Al final te doy una recomendación según cada caso de uso.

Matriz de Decisión Rápida

# Método	CSV	Excel (.xlsx)	Requiere Java	Requiere APEX	Ideal para
1 External Table	 Excelente		No	No	Cargas grandes y repetitivas
2 UTL_FILE + PL/SQL	 Bueno		No	No	Control total, archivos medianos
3 Text Importer de PL/SQL Developer	 Excelente		No	No	Cargas ad-hoc rápidas (GUI)
4 APEX_DATA_PARSER	 Excelente	 Excelente	No	Sí (5.1.4+)	Mejor opción general
5 Java Stored Procedure + POI	 Excelente		Sí	No	Cuando no hay APEX
6 SQL*Loader	 Excelente		No	No	Cargas masivas batch

Método 1: External Table (el más limpio para CSV)

Permite consultar un CSV como si fuera una tabla Oracle. **No carga los datos**, los lee directamente del archivo. Muy útil si el CSV se regenera periódicamente.

Paso 1: Crear el DIRECTORY (como SYS)

```
sql

-- En Windows, la ruta debe ser accesible por el usuario del servicio Oracle
CREATE OR REPLACE DIRECTORY DIR_CSV AS 'C:\oracle\csv_files';
GRANT READ, WRITE ON DIRECTORY DIR_CSV TO TU_ESQUEMA;
```

Paso 2: Colocar el archivo CSV en esa ruta

Ejemplo `clientes.csv`:

```
csv

id_cliente,identificacion,nombre_cliente,email,telefono,id_punto_venta
500,'900111222','Distribuidora ABC','abc@mail.com','3001112233',10000
501,'900333444','Comercial XYZ','xyz@mail.com','3004445566',10001
502,'900555666','Tienda 123','t123@mail.com','3007778899',10002
```

Paso 3: Crear la External Table

```
sql

CREATE TABLE ext_clientes_csv (
    id_cliente      NUMBER,
    identificacion  VARCHAR2(20),
    nombre_cliente  VARCHAR2(200),
    email           VARCHAR2(150),
    telefono        VARCHAR2(20),
    id_punto_venta  NUMBER
)
ORGANIZATION EXTERNAL (
    TYPE ORACLE_LOADER
    DEFAULT DIRECTORY DIR_CSV
    ACCESS PARAMETERS (
        RECORDS DELIMITED BY NEWLINE
        SKIP 1                                -- Salta la cabecera
        FIELDS TERMINATED BY ','
        OPTIONALLY ENCLOSED BY '"'          -- Soporta comillas
        MISSING FIELD VALUES ARE NULL
    )
```

```

        id_cliente,
        identificacion,
        nombre_cliente,
        email,
        telefono,
        id_punto_venta
    )
)
LOCATION ('clientes.csv')
)
REJECT LIMIT UNLIMITED;

```

Paso 4: Consultar y cargar en tabla real

```

sql

-- Ver contenido
SELECT * FROM ext_clientes_csv;

-- Cargar a tabla destino
INSERT INTO cliente (id_cliente, identificacion, nombre_cliente, email, telefono,
id_punto_venta)
SELECT id_cliente, identificacion, nombre_cliente, email, telefono, id_punto_venta
FROM ext_clientes_csv
WHERE id_cliente IS NOT NULL;

COMMIT;

```

Ventajas: Sin PL/SQL, muy rápido, auditable, reutilizable.

Desventajas: El archivo debe estar en el servidor DB, solo CSV.

Método 2: UTL_FILE + PL/SQL (control total)

Útil si el archivo viene de un proceso externo y quieres cargarlo bajo demanda desde PL/SQL Developer.

Procedimiento genérico

sql

```
CREATE OR REPLACE PROCEDURE pr_cargar_csv_clientes (
    p_nombre_archivo IN VARCHAR2
) IS
    v_archivo    UTL_FILE.FILE_TYPE;
    v_linea      VARCHAR2(32767);
    v_id         NUMBER;
    v_ident      VARCHAR2(20);
    v_nombre     VARCHAR2(200);
    v_email      VARCHAR2(150);
    v_tel        VARCHAR2(20);
    v_id_pv      NUMBER;
    v_contador   NUMBER := 0;
BEGIN
    v_archivo := UTL_FILE.FOPEN('DIR_CSV', p_nombre_archivo, 'R', 32767);

    -- Saltar cabecera
    BEGIN
        UTL_FILE.GET_LINE(v_archivo, v_linea);
    EXCEPTION WHEN NO_DATA_FOUND THEN NULL;
    END;

    LOOP
        BEGIN
            UTL_FILE.GET_LINE(v_archivo, v_linea);
        EXCEPTION
            WHEN NO_DATA_FOUND THEN EXIT;
        END;

        -- Parseo básico por comas (sin comillas)
        v_id      := TO_NUMBER(REGEXP_SUBSTR(v_linea, '^[^,]+', 1, 1));
        v_ident   := REGEXP_SUBSTR(v_linea, '^[^,]+', 1, 2);
        v_nombre  := REGEXP_SUBSTR(v_linea, '^[^,]+', 1, 3);
        v_email   := REGEXP_SUBSTR(v_linea, '^[^,]+', 1, 4);
        v_tel     := REGEXP_SUBSTR(v_linea, '^[^,]+', 1, 5);
        v_id_pv   := TO_NUMBER(REGEXP_SUBSTR(v_linea, '^[^,]+', 1, 6));
```

```

        INSERT INTO cliente (id_cliente, identificacion, nombre_cliente, email, telefono,
id_punto_venta)
        VALUES (v_id, v_ident, v_nombre, v_email, v_tel, v_id_pv);

        v_contador := v_contador + 1;
    END LOOP;

    UTL_FILE.FCLOSE(v_archivo);
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Filas cargadas: ' || v_contador);
EXCEPTION
    WHEN OTHERS THEN
        IF UTL_FILE.IS_OPEN(v_archivo) THEN
            UTL_FILE.FCLOSE(v_archivo);
        END IF;
        ROLLBACK;
        RAISE;
END pr_cargar_csv_clientes;
/

-- Ejecutar
EXEC pr_cargar_csv_clientes('clientes.csv');
```

Nota: Para CSV con campos entre comillas que contengan comas, debes mejorar el parser (usa una función split o `APEX_UTIL.STRING_TO_TABLE`, o `DBMS_UTILITY.COMMA_TO_TABLE`).

Método 3: Text Importer de PL/SQL Developer (el más rápido, GUI)

Como ya usas **PL/SQL Developer (Allround Automations)**, tienes una herramienta integrada que evita todo el código:

1. Menú **Tools** → **Text Importer**
2. **Data tab** → **Open data file...** → selecciona tu CSV
3. Configura:

- Delimiter:** , (o ;, o tabulación)

- Text qualifier:** "

- First row is header:** ✓

4.Pestaña **Columns**: revisa que cada campo se mapee bien

5.Pestaña **Data to Oracle**:

- Owner / Table:** tu esquema y tabla destino

- Mapea cada columna del CSV a la columna Oracle

6.Botón **Import**

Ventajas: Cero código, ideal para cargas puntuales.

Desventajas: Manual, no automatizable.

Método 4: APEX_DATA_PARSER (el más moderno, soporta Excel .xlsx real)

Desde **Oracle 12.2** y **APEX 5.1.4+**, Oracle incluye **APEX_DATA_PARSER** que lee **CSV, XLSX, JSON y XML** sin Java, sin POI, sin nada externo. Es la mejor opción si tienes APEX instalado (aunque no uses APEX para aplicaciones).

Ejemplo: Leer un XLSX y cargarlo en tabla

```
sql

CREATE OR REPLACE PROCEDURE pr_cargar_excel_clientes (
    p_nombre_archivo IN VARCHAR2
) IS
    v_blob      BLOB;
    v_archivo   BFILE;
BEGIN
    -- Leer el archivo Excel desde el directorio Oracle
    v_archivo := BFILENAME('DIR_CSV', p_nombre_archivo);
    DBMS_LOB.FILEOPEN(v_archivo);
    DBMS_LOB.LOADFROMFILE(v_blob, v_archivo, DBMS_LOB.GETLENGTH(v_archivo));
    DBMS_LOB.FILECLOSE(v_archivo);
```

```

-- Parsear y cargar directamente
INSERT INTO cliente (id_cliente, identificacion, nombre_cliente, email, telefono,
id_punto_venta)
SELECT TO_NUMBER(c001) AS id_cliente,
       c002           AS identificacion,
       c003           AS nombre_cliente,
       c004           AS email,
       c005           AS telefono,
       TO_NUMBER(c006) AS id_punto_venta
FROM TABLE(
    APEX_DATA_PARSER.PARSE(
        p_content => v_blob,
        p_file_name => p_nombre_archivo,
        p_skip_rows => 1      -- salta cabecera
    )
)
WHERE c001 IS NOT NULL;

COMMIT;
DBMS_OUTPUT.PUT_LINE('Filas insertadas: ' || SQL%ROWCOUNT);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END pr_cargar_excel_clientes;
/

```

Preview del contenido antes de cargar

```

sql

SELECT c001 AS id_cliente,
       c002 AS identificacion,
       c003 AS nombre_cliente,
       c004 AS email,
       c005 AS telefono,
       c006 AS id_punto_venta
FROM TABLE(
    APEX_DATA_PARSER.PARSE(
        p_content => (SELECT b FROM (SELECT BFILENAME('DIR_CSV','clientes.xlsx') b
FROM dual)),

```

```
p_file_name => 'clientes.xlsx',  
p_skip_rows => 1  
)  
);
```

Ventajas:

- Lee **XLSX real** (no requiere convertir a CSV)
- Detecta automáticamente tipos, separadores, hojas
- Sin Java, sin instalaciones extra
- Muy rápido

Desventajas: Requiere APEX instalado en la BD.

Método 5: Java Stored Procedure con Apache POI (si no hay APEX)

Si no tienes APEX y necesitas leer Excel binario real, puedes cargar la librería Apache POI en la BD y exponer un procedimiento Java.

Paso 1: Cargar los JARs de POI en la BD (como SYS)

```
sql  
  
-- Cargar Las clases Java de POI en La BD  
CALL dbms_java.loadjava('-resolve -grant PUBLIC -file C:\poi\poi-5.2.3.jar');  
CALL dbms_java.loadjava('-resolve -grant PUBLIC -file C:\poi\poi-ooxml-5.2.3.jar');  
CALL dbms_java.loadjava('-resolve -grant PUBLIC -file C:\poi\xmlbeans-5.1.1.jar');  
CALL dbms_java.loadjava('-resolve -grant PUBLIC -file C:\poi\commons-collections4-4.4.jar');  
CALL dbms_java.loadjava('-resolve -grant PUBLIC -file C:\poi\commons-compress-1.22.jar');
```

Paso 2: Crear el wrapper Java

```
java  
  
CREATE OR REPLACE AND COMPILE JAVA SOURCE NAMED "ExcelReader" AS
```

```

import java.io.*;
import java.sql.*;
import oracle.sql.*;
import oracle.jdbc.driver.*;
import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public class ExcelReader {
    public static void cargarClientes(String rutaArchivo) throws Exception {
        Connection conn = new OracleDriver().defaultConnection();
        FileInputStream fis = new FileInputStream(rutaArchivo);
        Workbook wb = new XSSFWorkbook(fis);
        Sheet hoja = wb.getSheetAt(0);
        PreparedStatement ps = conn.prepareStatement(
            "INSERT INTO cliente (id_cliente, identificacion, nombre_cliente, email,
telefono, id_punto_venta) " +
            "VALUES (?, ?, ?, ?, ?, ?)");

        for (int i = 1; i <= hoja.getLastRowNum(); i++) { // desde 1, salta cabecera
            Row fila = hoja.getRow(i);
            if (fila == null) continue;
            ps.setInt(1, (int) fila.getCell(0).getNumericCellValue());
            ps.setString(2, fila.getCell(1).getStringCellValue());
            ps.setString(3, fila.getCell(2).getStringCellValue());
            ps.setString(4, fila.getCell(3).getStringCellValue());
            ps.setString(5, fila.getCell(4).getStringCellValue());
            ps.setInt(6, (int) fila.getCell(5).getNumericCellValue());
            ps.executeUpdate();
        }
        ps.close();
        wb.close();
        fis.close();
    }
}
/

```

Paso 3: Crear el procedimiento PL/SQL que llama al Java

sql

```
CREATE OR REPLACE PROCEDURE pr_cargar_excel_java (
```

```
p_ruta IN VARCHAR2
) AS
LANGUAGE JAVA NAME 'ExcelReader.cargarClientes(java.lang.String)';
/
```

Paso 4: Ejecutar

```
sql

BEGIN
    pr_cargar_excel_java('C:\oracle\csv_files\clientes.xlsx');
    COMMIT;
END;
/
```

Ventajas: Lee Excel binario real, sin APEX.

Desventajas: Requiere cargar JARs, permisos `dbms_java`, Java habilitado en la BD, más complejo de mantener.

Método 6: SQL*Loader (cargas masivas)

El clásico. Se ejecuta desde línea de comandos (Windows).

Archivo de control `carga_clientes.ctl`

```
text

LOAD DATA
INFILE 'C:\oracle\csv_files\clientes.csv'
BADFILE 'C:\oracle\csv_files\clientes.bad'
DISCARDFILE 'C:\oracle\csv_files\clientes.dsc'
APPEND
INTO TABLE cliente
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
TRAILING NULLCOLS
(
    id_cliente,
```

```
    identificacion,  
    nombre_cliente,  
    email,  
    telefono,  
    id_punto_venta  
)
```

Ejecución

```
cmd  
  
sqlldr userid=tu_esquema/tu_password@tu_servicio control=carga_clientes.ctl  
log=carga_clientes.log
```

Ventajas: Extremadamente rápido para millones de filas.

Desventajas: Solo CSV, requiere acceso al servidor, no es PL/SQL.

Recomendación según tu caso

Escenario	Método recomendado
Carga puntual desde PL/SQL Developer	Text Importer (GUI, Método 3)
Carga recurrente de CSV en servidor DB	External Table (Método 1)
Carga recurrente desde cliente (no servidor)	UTL_FILE o SQL*Loader
Necesitas leer Excel .xlsx real	APEX_DATA_PARSER (Método 4)
No hay APEX pero necesitas .xlsx	Java + POI (Método 5)
Millones de registros	SQL*Loader (Método 6)

Ejemplo Integrador: Cargar Excel en la tabla cliente y validar con la jerarquía

Combinando lo mejor de cada enfoque, un procedimiento robusto para tu caso (cargar clientes desde Excel y validar que el PV existe en la división correcta):

sql

```
CREATE OR REPLACE PROCEDURE pr_cargar_clientes_excel (
    p_nombre_archivo IN VARCHAR2
) IS
    v_blob      BLOB;
    v_archivo    BFILE;
    v_insertados NUMBER := 0;
    v_rechazados NUMBER := 0;
BEGIN
    v_archivo := BFILENAME('DIR_CSV', p_nombre_archivo);
    DBMS_LOB.FILEOPEN(v_archivo);
    DBMS_LOB.LOADFROMFILE(v_blob, v_archivo, DBMS_LOB.GETLENGTH(v_archivo));
    DBMS_LOB.FILECLOSE(v_archivo);

    FOR r IN (
        SELECT c001, c002, c003, c004, c005, c006
        FROM TABLE(
            APEX_DATA_PARSER.PARSE(
                p_content => v_blob,
                p_file_name => p_nombre_archivo,
                p_skip_rows => 1
            )
        )
        WHERE c001 IS NOT NULL
    ) LOOP
        BEGIN
            -- Validar que el punto de venta pertenezca a 'Clientes de Mi Negocio'
            DECLARE
                v_valido NUMBER;
            BEGIN
                SELECT COUNT(*) INTO v_valido
                FROM punto_venta pv
                JOIN agencia a ON a.id_agencia = pv.id_agencia
```

```

        JOIN territorio t ON t.id_territorio = a.id_territorio
        JOIN division d ON d.id_division = t.id_division
        WHERE pv.id_punto_venta = TO_NUMBER(r.c006)
              AND d.nombre_division = 'Clientes de Mi Negocio';

        IF v_valido = 0 THEN
            v_rechazados := v_rechazados + 1;
            DBMS_OUTPUT.PUT_LINE('PV ' || r.c006 || ' no pertenece a la división.
Fila rechazada.');
```

CONTINUE;

```

        END IF;
    END;

    INSERT INTO cliente (id_cliente, identificacion, nombre_cliente, email,
telefono, id_punto_venta)
    VALUES (
        TO_NUMBER(r.c001),
        r.c002,
        r.c003,
        r.c004,
        r.c005,
        TO_NUMBER(r.c006)
    );
    v_insertados := v_insertados + 1;

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        v_rechazados := v_rechazados + 1;
        DBMS_OUTPUT.PUT_LINE('Cliente duplicado: ' || r.c001);
    WHEN OTHERS THEN
        v_rechazados := v_rechazados + 1;
        DBMS_OUTPUT.PUT_LINE('Error fila ' || r.c001 || ': ' || SQLERRM);

    END;
END LOOP;

COMMIT;
DBMS_OUTPUT.PUT_LINE('Insertados: ' || v_insertados || ' | Rechazados: ' ||
v_rechazados);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;

```

```
RAISE;  
END pr_cargar_clientes_excel;  
/
```

Preguntas para afinar la solución

1. **¿Tienes Oracle APEX instalado?** (Verifica con `SELECT * FROM dba_registry WHERE comp_id='APEX';`). Si sí, vamos directo por `APEX_DATA_PARSER`.
2. **¿El archivo Excel/CSV viene de un usuario final o de un proceso batch?** Si viene de un usuario, conviene un formulario Oracle Forms con `CLIENT_TEXT_IO` o una carga por APEX.
3. **¿Qué versión de Oracle?** 12.2+ habilita `APEX_DATA_PARSER`; 11g requiere Java+POI o convertir a CSV.
4. **¿El archivo está en el servidor DB o en el PC del usuario?** Si está en el PC, no puedes usar `UTL_FILE` ni `BFILE` directamente (necesitas subirlo primero o usar Forms `CLIENT_TEXT_IO`).
5. **¿Cuántas filas por archivo?** Menos de 10k → cualquier método. Más de 100k → External Table o SQL*Loader.

Dime cuál de estos escenarios aplica y te armo el flujo exacto end-to-end (por ejemplo: formulario Oracle Forms que sube el Excel + procedimiento PL/SQL que lo parsea con `APEX_DATA_PARSER` + validación + carga).

Sí. En tu entorno **Oracle Database + PL/SQL Developer + Windows**, hay varias formas. Para un proceso empresarial yo distinguiría entre **CSV** y **Excel .xlsx**, porque Oracle no los trata igual.

1. La opción que recomiendo: CSV + UTL_FILE

Si puedes elegir el formato de entrada, **usa CSV**. Es mucho más sencillo y robusto para Oracle.

El flujo sería:

```
archivo.csv  
↓  
carpeta del servidor Oracle  
↓  
DIRECTORY de Oracle  
↓  
UTL_FILE  
↓  
PL/SQL  
↓  
tabla temporal/staging  
↓  
validación  
↓
```

tabla definitiva

Por ejemplo, tienes:

clientes.csv

con:

```
ID_CLIENTE, NOMBRE, TELEFONO, SALDO
1001, Juan Perez, 88881111, 150.00
1002, Maria Lopez, 88882222, 300.00
1003, Pedro Gomez, 88883333, 0
```

2. Crear la tabla destino

Por ejemplo:

```
CREATE TABLE CLIENTES_IMPORTADOS (
    ID_CLIENTE NUMBER,
    NOMBRE VARCHAR2(200),
    TELEFONO VARCHAR2(30),
    SALDO NUMBER(15,2),
    FECHA_CARGA DATE DEFAULT SYSDATE
);
```

Pero **yo recomiendo crear primero una tabla STAGING:**

```
CREATE TABLE STG_CLIENTES (
    ID_CLIENTE VARCHAR2(50),
    NOMBRE VARCHAR2(200),
    TELEFONO VARCHAR2(50),
    SALDO VARCHAR2(50),
    FECHA_CARGA DATE DEFAULT SYSDATE
);
```

¿Por qué VARCHAR2?

Porque primero recibimos el archivo "tal cual" y después validamos.

Por ejemplo, si alguien coloca:

SALDO = ABC

no queremos que falle toda la carga.

3. Crear DIRECTORY

Supongamos que en Windows Oracle tendrá:

C:\ORACLE\IMPORTACIONES

El DBA ejecutaría:

```
CREATE OR REPLACE DIRECTORY DIR_IMPORTACIONES  
AS 'C:\ORACLE\IMPORTACIONES';
```

Luego:

```
GRANT READ, WRITE  
ON DIRECTORY DIR_IMPORTACIONES  
TO MI_USUARIO;
```

Importante

La carpeta debe existir **en el servidor donde está instalado Oracle Database**, no necesariamente en la PC donde tienes PL/SQL Developer.

Esto confunde bastante.

Si tienes:

```
PC usuario  
  ↓  
PL/SQL Developer  
  ↓  
Oracle Database Server
```

UTL_FILE busca:

C:\ORACLE\IMPORTACIONES

en el **servidor Oracle**, no en tu computadora.

4. Leer el CSV

Podemos crear:

```
CREATE OR REPLACE PROCEDURE PRC_IMPORTAR_CLIENTES  
IS  
    V_FILE UTL_FILE.FILE_TYPE;  
  
    V_LINEA VARCHAR2(32767);  
  
    V_ID_CLIENTE VARCHAR2(50);  
    V_NOMBRE     VARCHAR2(200);  
    V_TELEFONO   VARCHAR2(50);  
    V_SALDO      VARCHAR2(50);  
  
BEGIN  
  
    V_FILE := UTL_FILE.FOPEN(  
        'DIR_IMPORTACIONES',  
        'clientes.csv',  
        'R',
```

```

32767
);

-- Saltar encabezado
UTL_FILE.GET_LINE(
    V_FILE,
    V_LINEA
);

LOOP

    BEGIN

        UTL_FILE.GET_LINE(
            V_FILE,
            V_LINEA
        );

        -- Separar columnas
        V_ID_CLIENTE := REGEXP_SUBSTR(
            V_LINEA,
            '^[^,]+' ,
            1,
            1
        );

        V_NOMBRE := REGEXP_SUBSTR(
            V_LINEA,
            '^[^,]+' ,
            1,
            2
        );

        V_TELEFONO := REGEXP_SUBSTR(
            V_LINEA,
            '^[^,]+' ,
            1,
            3
        );

        V_SALDO := REGEXP_SUBSTR(
            V_LINEA,
            '^[^,]+' ,
            1,
            4
        );

        INSERT INTO STG_CLIENTES (
            ID_CLIENTE,
            NOMBRE,
            TELEFONO,
            SALDO
        )
        VALUES (
            V_ID_CLIENTE,
            V_NOMBRE,
            V_TELEFONO,
            V_SALDO
        );

```

```

        EXCEPTION
            WHEN NO_DATA_NEEDED THEN
                EXIT;

            WHEN OTHERS THEN
                DBMS_OUTPUT.PUT_LINE(
                    'Error en línea: ' || V_LINEA
                );
        END;

    END LOOP;

    UTL_FILE.FCLOSE(V_FILE);

    COMMIT;

EXCEPTION

    WHEN UTL_FILE.INVALID_PATH THEN

        RAISE_APPLICATION_ERROR(
            -20001,
            'La ruta DIR_IMPORTACIONES no existe o no tiene permisos.'
        );

    WHEN UTL_FILE.INVALID_MODE THEN

        RAISE_APPLICATION_ERROR(
            -20002,
            'Modo de lectura inválido.'
        );

    WHEN OTHERS THEN

        IF UTL_FILE.IS_OPEN(V_FILE) THEN
            UTL_FILE.FCLOSE(V_FILE);
        END IF;

        ROLLBACK;

        RAISE;

END;
/

```

Pero hay una mejora importante: **ese ejemplo de separación por comas es adecuado solamente para CSV sencillos.**

Por ejemplo:

1001, Juan Perez, 88881111, 150

pero puede fallar con:

1001, "Perez, Juan", 88881111, 150

porque la coma forma parte del nombre.

Para producción utilizaría otro mecanismo de parsing o un formato CSV controlado.

5. Después validamos los datos

Una de las ventajas de usar STG_CLIENTES es que podemos hacer:

```
SELECT *
FROM STG_CLIENTES
WHERE NOT REGEXP_LIKE(SALDO, '^[0-9]+(\.[0-9]+)?$');
```

Encontraríamos:

ID_CLIENTE	NOMBRE	SALDO
1005	Pedro López	ABC

Y podemos detectar:

```
SELECT *
FROM STG_CLIENTES
WHERE ID_CLIENTE IS NULL;
```

etc.

6. Pasar los datos buenos a la tabla definitiva

Por ejemplo:

```
INSERT INTO CLIENTES_IMPORTADOS (
    ID_CLIENTE,
    NOMBRE,
    TELEFONO,
    SALDO
)
SELECT
    TO_NUMBER(ID_CLIENTE),
    TRIM(NOMBRE),
    TRIM(TELEFONO),
    TO_NUMBER(SALDO)
FROM STG_CLIENTES
WHERE REGEXP_LIKE(
    ID_CLIENTE,
    '^[0-9]+$'
)
AND REGEXP_LIKE(
    SALDO,
    '^[0-9]+(\.[0-9]+)?$'
);

COMMIT;
```

7. Incluso puedes usar MERGE

Si el CSV contiene clientes que **pueden existir o no existir**, esto es mucho mejor.

```
MERGE INTO CLIENTES C
USING (
    SELECT
        TO_NUMBER(ID_CLIENTE) ID_CLIENTE,
        TRIM(NOMBRE) NOMBRE,
        TRIM(TELEFONO) TELEFONO
    FROM STG_CLIENTES
    WHERE REGEXP_LIKE(ID_CLIENTE, '^[0-9]+$')
) S
ON (
    C.ID_CLIENTE = S.ID_CLIENTE
)

WHEN MATCHED THEN

    UPDATE SET
        C.NOMBRE = S.NOMBRE,
        C.TELEFONO = S.TELEFONO

WHEN NOT MATCHED THEN

    INSERT (
        ID_CLIENTE,
        NOMBRE,
        TELEFONO
    )
    VALUES (
        S.ID_CLIENTE,
        S.NOMBRE,
        S.TELEFONO
    );

COMMIT;
```

Esto permite:

CSV

↓

```
Cliente existe?
├── SI → UPDATE
└── NO → INSERT
```

8. ¿Y si el archivo es Excel .xlsx?

Aquí cambia la situación.

Oracle **no tiene un equivalente sencillo a UTL_FILE** para leer directamente un **.xlsx** moderno.

Un **.xlsx** internamente es un paquete ZIP con varios archivos XML.

Por eso **no recomiendo intentar leer Excel directamente con PL/SQL** si tienes control sobre el proceso.

Tienes varias alternativas.

9. Alternativa Excel → CSV → Oracle

Para tu entorno Windows, esta sería probablemente mi opción favorita:

```
Excel
↓
Guardar como CSV
↓
C:\ORACLE\IMPORTACIONES
↓
Oracle
↓
UTL_FILE
↓
STG
↓
Tabla
```

Es simple, fácil de mantener y fácil de auditar.

10. Si obligatoriamente debe ser .xlsx

Entonces usaría una herramienta externa.

Por ejemplo:

```
Excel
↓
Python / PowerShell
↓
leer XLSX
↓
validar
↓
Oracle
```

Con Python podrías utilizar:

```
pandas
openpyxl
oracledb
```

El proceso sería:

```
archivo.xlsx
↓
Python
↓
pandas.read_excel()
↓
validación
↓
Oracle
↓
STG_CLIENTES
↓
MERGE
```

Esto es especialmente interesante en tu caso porque estás trabajando en **Windows**.

11. Otra opción: Oracle External Table

Si la carga es principalmente CSV, hay otra alternativa bastante interesante:

Oracle External Table.

Por ejemplo:

```
CREATE TABLE EXT_CLIENTES (
    ID_CLIENTE NUMBER,
    NOMBRE VARCHAR2(200),
    TELEFONO VARCHAR2(50),
    SALDO NUMBER(15,2)
)
ORGANIZATION EXTERNAL (
    TYPE ORACLE_LOADER
    DEFAULT DIRECTORY DIR_IMPORTACIONES
    ACCESS PARAMETERS (
        RECORDS DELIMITED BY NEWLINE
        FIELDS TERMINATED BY ','
        OPTIONALLY ENCLOSED BY '"'
        MISSING FIELD VALUES ARE NULL
    )
    LOCATION ('clientes.csv')
)
REJECT LIMIT UNLIMITED;
```

Entonces puedes hacer:

```
SELECT *
FROM EXT_CLIENTES;
```

¡Sin tener que hacer un `UTL_FILE.GET_LINE` manual!

Y posteriormente:

```

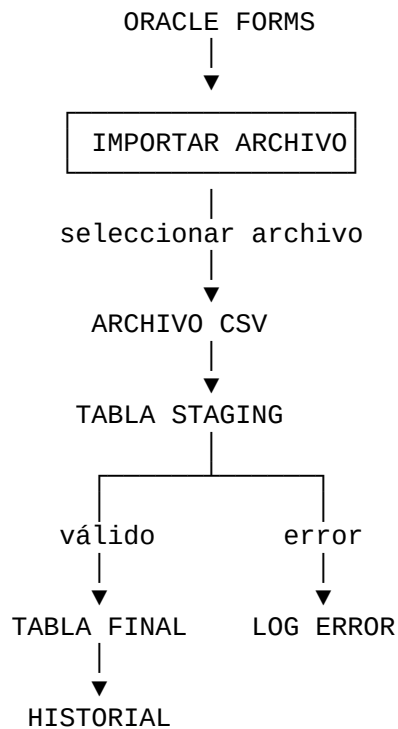
INSERT INTO CLIENTES_IMPORTADOS (
    ID_CLIENTE,
    NOMBRE,
    TELEFONO,
    SALDO
)
SELECT
    ID_CLIENTE,
    NOMBRE,
    TELEFONO,
    SALDO
FROM EXT_CLIENTES;

```

Para **cargas masivas de CSV**, esta opción puede ser incluso mejor.

12. Para tu sistema, yo usaría este patrón

Como vienes trabajando con Oracle Forms y procesos empresariales, te recomiendo diseñar un **módulo genérico de importación**:



Por ejemplo, crear:

```

IMPORTACIONES
-----
ID_IMPORTACION
NOMBRE_ARCHIVO
FECHA_IMPORTACION
USUARIO
TIPO_ARCHIVO

```

REGISTROS_LEIDOS
REGISTROS_OK
REGISTROS_ERROR
ESTADO

Y:

IMPORTACION_ERRORES

ID_ERROR
ID_IMPORTACION
NUMERO_LINEA
DATOS_LINEA
MENSAJE_ERROR
FECHA_ERROR

Entonces desde Forms puedes mostrar:

IMPORTACIÓN DE CLIENTES	
Archivo: clientes_2026.csv	
[Seleccionar archivo]	
Registros:	2,500
Correctos:	2,480
Errores:	20
[VALIDAR]	[IMPORTAR]

Y al finalizar:

Importación completada

Archivo: clientes_2026.csv

Registros leídos: 2,500
Registros importados: 2,480
Registros con error: 20

[Ver errores]

¿Cuál elegiría para tu caso?

Método	CSV	Excel	Facilidad	Rendimiento	Recomendación
UTL_FILE	✓	✗	Alta	Bueno	★★★★
External Table	✓	✗	Media	Excelente	★★★★★
Python + openpyxl	✓	✓	Media	Excelente	★★★★★
PL/SQL intentando leer XLSX	✗	⚠	Baja	Variable	✗
Excel → CSV → Oracle	✓	✓*	Muy alta	Bueno	★★★★★

* El usuario guarda el Excel como CSV.

Para un sistema Oracle Forms empresarial

Yo haría:

Si el usuario puede trabajar con CSV:

```
Oracle Forms
  ↓
CSV
  ↓
External Table / STAGING
  ↓
validación
  ↓
MERGE
  ↓
tabla definitiva
```

Si el requisito es obligatoriamente Excel:

```
Oracle Forms
  ↓
.xlsx
  ↓
Python en Windows
  ↓
STAGING Oracle
  ↓
validación
  ↓
MERGE
```

Y si vas a importar **miles o cientos de miles de registros**, prefiero claramente **External Tables + tabla STAGING** antes que recorrer línea por línea con UTL_FILE.

Si me dices **qué versión de Oracle tienes (11g/12c/19c/etc.)**, puedo darte el ejemplo completo de un **módulo Oracle Forms para seleccionar un CSV, cargarlo a una tabla STAGING, validar errores, mostrar los errores en un bloque Forms y finalmente hacer el MERGE a la tabla definitiva**, que sería una solución bastante apropiada para el entorno que estás desarrollando.

Sí. Si lo que quieres es **llevar ese PDF a Excel sin instalar ningún programa externo**, hay varias opciones.

Para tu caso concreto —PDF con una tabla de **2 campos**, donde cada fila tiene algo como:

```
Nombre de empresa 10 días
Nombbre de la empresa 2 30 días
```

Nombre de la empresa 3 15 días

y puede existir texto antes de la tabla— la opción más sencilla sería **usar Excel directamente**, sin Python ni otro programa.

Opción 1: Excel + Power Query

Si tienes una versión de Excel que incluye **Power Query**, puedes hacer:

Datos → Obtener datos → De archivo → De PDF

Excel analizará el PDF y te mostrará las tablas detectadas.

El problema es que en tu PDF probablemente Excel no detectará correctamente las dos columnas porque **no existe una separación física clara entre Empresa y Días**.

Pero podemos resolverlo dentro de Excel.

Por ejemplo, después de importar el texto:

Texto original

Nombre de empresa 10 días

Nombbre de la empresa 2 30 días

Nombre de la empresa 3 15 días

En Power Query podemos crear:

Empresa

Nombre de empresa

Nombbre de la empresa 2

Nombre de la empresa 3

y Días

10

30

15

La regla sería:

Buscar el número que aparece inmediatamente antes de **días**; ese número es **Días** y todo lo anterior es **Empresa**.

Esto además permite ignorar automáticamente párrafos anteriores que no tengan ese patrón.

Opción 2: Copiar directamente desde el PDF a Excel

Para un PDF sencillo como el tuyo, incluso puedes:

1. Abrir el PDF.
2. Seleccionar todo el texto.

3. Copiar.
4. Pegar en Excel.
5. Usar una fórmula para separar los campos.

Por ejemplo, si todo queda en la columna A:

Nombre de empresa 10 días
Nombbre de la empresa 2 30 días
Nombre de la empresa 3 15 días

Puedes extraer los días con una fórmula.

Si A2 contiene:

Nombre de empresa 10 días

podrías utilizar:

```
=VALOR(TEXTOANTES(TEXTO.DESPUES(A2;" ");" días"))
```

Aunque para tu caso es mejor utilizar una fórmula que busque específicamente el número antes de " días", porque el nombre de la empresa puede contener números.

Opción 3: Excel + Power Query es la que más te recomiendo

La estructura sería:

```
PDF
↓
Excel / Power Query
↓
Extraer texto
↓
Detectar filas que tengan "número + días"
↓
Ignorar párrafos anteriores
↓
Separar
├── Empresa
└── Días
↓
Tabla Excel
```

Y no necesitas instalar Python, Java, Adobe Acrobat ni ningún programa adicional.

Además, podemos hacer que quede preparado para que después simplemente coloques otro PDF y presiones **Actualizar**, y Excel vuelva a generar la tabla.

Y si tu objetivo final es Oracle

Incluso podemos hacer el flujo:

PDF

↓
Excel / Power Query
↓
Empresa | Dias
↓
Guardar como CSV
↓
Oracle
↓
Tabla temporal
↓
Validación
↓
Tabla definitiva

Así tampoco necesitarías un programa externo para la transformación del PDF.

Si quieres, te puedo dar **el procedimiento exacto en Excel Power Query para tu PDF**, incluyendo la fórmula/regla para detectar automáticamente Empresa y Dias aunque haya varios párrafos antes de la tabla.